

COPA DO MUNDO 2026

Aplicação Web Completa — Do Zero ao Deploy

Relatório Técnico do Projeto

Backend em Python/FastAPI · Frontend em React + Vite + Tailwind CSS
Banco de dados SQLite local · Desenvolvimento 100% assistido por IA (Claude Code)

Setembro de 2026

github.com/fernandojsmelo/copa2026

Sumário

- 1 Visão Geral do Projeto
- 2 Stack Tecnológica
- 3 Metodologia de Desenvolvimento
- 4 As Sprints: o que foi construído
- 5 Decisões de Engenharia e Resolução de Inconsistências
- 6 Funcionalidades Principais
- 7 Qualidade e Testes
- 8 Deploy e Infraestrutura
- 9 Estrutura do Repositório
- 10 Status Final

1. Visão Geral do Projeto

O **Copa 2026** é uma aplicação web completa dedicada à Copa do Mundo FIFA 2026, sediada nos Estados Unidos, Canadá e México — a 23ª edição do torneio, com 48 seleções, 12 grupos de 4 equipes e 104 partidas ao todo. A plataforma cobre, em português brasileiro, do acompanhamento em tempo real dos jogos até um simulador completo de bolão com chaveamento de mata-mata.

Objetivos de negócio

- Ser referência de informação sobre a Copa 2026 em português.
- Engajar usuários com funcionalidades interativas — bolão e escalação da Seleção Brasileira.
- Oferecer experiência fluida tanto em mobile quanto em desktop.
- Entregar uma plataforma de alta performance, com dados precisos e bem estruturados.

Público-alvo

Torcedores brasileiros e entusiastas do futebol, participantes de bolões esportivos, e jornalistas/analistas esportivos — faixa etária de 16 a 55 anos.

Princípio de arquitetura central

Uma decisão deliberada orienta todo o projeto: **nenhum Docker, nenhum servidor externo**. O backend roda direto na máquina com um único arquivo SQLite como banco de dados — simplicidade, zero configuração e portabilidade total, à custa de escalabilidade horizontal, que não é um requisito deste projeto.

2. Stack Tecnológica

Camada	Tecnologia	Motivo da escolha
Backend	Python 3.12 + FastAPI + Uvicorn	Tipagem, performance e documentação automática (OpenAPI/Swagger).
Banco de dados	SQLite (SQL puro, sem ORM)	Controle total das queries, sem overhead, fácil de depurar; modo WAL em produção.
Frontend	React 19 + Vite	Build rápido, HMR nativo, ecossistema maduro.
Estilização	Tailwind CSS v3	Velocidade de desenvolvimento e consistência visual via tema customizado.
Roteamento	React Router v6	Padrão de mercado para SPAs em React.
Estado global	Zustand	Leveza para estado simples (sessão, tema, autenticação admin) sem boilerplate.
Drag & Drop	HTML5 Drag and Drop API	Zero dependência extra para a escalação interativa.
Exportação de imagens	html2canvas	Única biblioteca externa relevante do frontend — carregada sob demanda (lazy).

Regras não negociáveis do projeto

- Nunca usar Docker — o projeto roda direto na máquina local.
- Nunca usar ORM — SQL puro, sempre parametrizado (proteção contra SQL injection).
- Nunca hardcodar SQL no frontend — toda lógica de banco fica no backend.
- Sessão do usuário via `crypto.randomUUID()`, salva em `sessionStorage`, sem login/senha para usuários comuns.
- Chave de administrador lida de variável de ambiente, nunca exposta no frontend.

3. Metodologia de Desenvolvimento

O projeto foi construído de forma **iterativa, sprint a sprint**, com o escopo e a ordem definidos em `Tasks.md`. Cada sprint seguiu o mesmo ciclo disciplinado:

- **Implementação do backend** — rotas, serviços e, quando necessário, migrations de banco.
- **Validação do backend isolada** — testes diretos contra a API antes de tocar no frontend.
- **Implementação do frontend** — componentes, páginas e hooks de dados.
- **Teste real no navegador** — fluxos exercitados de ponta a ponta via automação de browser, em desktop e em viewport mobile, sempre conferindo o console por erros.
- **Atualização da documentação** — `Tasks.md` e `README.md` marcados e anotados a cada entrega.
- **Commits pequenos e coesos** — cada commit cobre uma mudança lógica (backend, frontend, docs), nunca uma mistura de assuntos não relacionados.
- **Confirmação explícita antes de cada `git push`** — nenhum envio ao repositório remoto sem aprovação direta.

Esse ciclo revelou, mais de uma vez, bugs reais que só apareciam sob teste de verdade — não apenas leitura de código — e cada um foi corrigido antes de seguir em frente (detalhes na seção 5).

4. As Sprints: o que foi construído

Nove sprints, do zero ao deploy. Abaixo, o resumo de cada uma — funcionalidade entregue, não a lista exaustiva de arquivos.

SPRINT 0 Fundação e Setup

- Estrutura de pastas (backend/frontend), banco SQLite com schema completo (seleções, jogos, jogadores, bolões, palpites, escalações) e migrations versionadas.
- Seeds: 48 seleções, 72 jogos da fase de grupos, elenco completo do Brasil (23 jogadores) e elencos placeholder das demais 47 seleções.
- Backend base: FastAPI com CORS, logging de requisições, tratamento global de erros, health check.
- Frontend base: React + Vite + Tailwind com o tema visual da marca, roteamento, Header/Footer/Layout.

SPRINT 1 Landing Page

- Hero com contagem regressiva até a abertura da Copa (11/06/2026, 17h de Brasília).
- Seção de estatísticas com animação de contagem ao entrar na viewport.
- Widgets de próximos jogos e últimos resultados, consumindo a API real.
- Navegação principal com menu hambúrguer responsivo e indicador de página ativa.

SPRINT 2 Tabela de Jogos

- Listagem paginada dos 104 jogos com filtros por fase, grupo, seleção e data.
- Agrupamento visual por data, skeleton de carregamento e estado vazio tratado.
- Página de detalhe de cada jogo.
- Endpoint administrativo para atualizar placar e status, autenticado.

SPRINT 3 Classificação e Grupos

- Serviço de cálculo de classificação aplicando os critérios de desempate da FIFA na ordem correta: pontos → saldo de gols → gols marcados → confronto direto (mini-liga) → ranking FIFA.
- Página de grupos com cores de classificado/em disputa/eliminado e seção de potes do sorteio.

SPRINT 4 Elencos das Seleções

- Perfil de cada seleção (bandeira, confederação, treinador, ranking FIFA, posição no grupo).
- Tabela de elenco filtrável por posição, com busca por nome e navegação entre seleções.

SPRINT 5 Escalação da Seleção Brasileira

- Campo de futebol interativo (SVG) com as 7 formações táticas oficiais do briefing.
- Escalação por arrastar-e-soltar no desktop e por toque (selecionar + posicionar) no mobile.
- Troca automática de jogadores ao mudar de formação, exportação da escalação como imagem (PNG) e persistência de até 10 escalações salvas por sessão.

SPRINT 6 Bolão e Simulador de Chaveamento

- Simulador da fase de grupos com classificação em tempo real conforme os palpites.
- Chaveamento do mata-mata calculado e cascadeado automaticamente — do vencedor das oitavas até o campeão — incluindo prorrogação e disputa de pênaltis quando o placar empata.
- Gestão de múltiplos bolões por sessão: criar, duplicar e remover.
- Auto-save com indicador de progresso ("X de 104 palpites preenchidos").

SPRINT 7 Painel Admin e Polimento

- Painel administrativo protegido por senha, para atualizar placares/status e editar elencos.
- Sistema de notificações (toasts), skeleton loaders, retry automático de rede, transições de página, scroll-to-top e meta tags dinâmicas por página para SEO.

SPRINT 8 Testes e Deploy

- Validação de cenários de classificação com desempate real, simulação completa de bolão, as 7 formações de escalação e a contagem regressiva até zero.
- Guia de deploy (systemd + nginx), script de backup automático do banco e modo WAL do SQLite ativado para melhor concorrência em produção.

5. Decisões de Engenharia e Resolução de Inconsistências

Um projeto real raramente tem uma especificação perfeita. Abaixo, os pontos em que os documentos de origem (PRD/Tasks) traziam dados incompletos, contraditórios ou simplesmente ausentes — e a decisão tomada em cada caso, sempre documentada no código e no README para rastreabilidade.

Seleções duplicadas no sorteio

A tabela de grupos do PRD trazia Colômbia e Croácia cada uma alocada em dois grupos diferentes ao mesmo tempo — logicamente impossível. As vagas conflitantes foram semeadas como placeholder "A Definir", junto com as vagas de repescagem genuinamente em aberto.

"48 jogos" vs. 72 jogos reais

PRD e Tasks citavam 48 jogos na fase de grupos, mas o round-robin padrão de 4 seleções por grupo (regra oficial da Copa) resulta matematicamente em 72 — é esse número que faz o total de 104 jogos do torneio fechar exatamente com o citado na seção 1.2 do PRD.

Chaveamento do mata-mata sem tabela oficial

O briefing pedia o cruzamento oficial das oitavas de final "hardcoded", mas não fornecia a tabela em si. Em vez de arriscar apresentar um cruzamento inventado como se fosse o oficial da FIFA, foi implementado um algoritmo próprio e claramente documentado no código.

Confrontos hipotéticos do bolão

Os confrontos de mata-mata simulados no bolão dependem de quem cada usuário classificou na fase de grupos — não existe uma linha "jogo real" compartilhada possível para eles. Uma migration tornou a referência a jogo opcional e introduziu um identificador de confronto virtual (ex.: "oitavas-1"), preservando a integridade do restante do schema.

Bug de concorrência no SQLite + FastAPI

Um erro intermitente ("SQLite objects created in a thread...") surgiu ao testar a API pelo navegador — o FastAPI executa dependências síncronas em threads diferentes do pool do anyio. Corrigido com `check_same_thread=False` na conexão.

Duplicação silenciosa de dados no seed

Rodar o script de inicialização do banco sem a flag de reset duplicava todos os dados a cada execução — um bug latente desde a primeira sprint, só exposto ao testar o fluxo de deploy. Corrigido com uma tabela de controle de migrations e seeds já aplicados.

pyproject.toml sem tabela [project] quebrava o build na Vercel

O build do backend na Vercel usa uv para resolver as dependências a partir do `pyproject.toml`, mas o arquivo só continha configuração de lint/format (`ruff`, `black`) — sem uma tabela `[project]`, o comando `uv lock` falhava com "No project table found". Corrigido adicionando `[project]` (nome, versão e as dependências espelhando `requirements.txt`) e `[tool.uv] package = false`, com o `uv.lock` resultante versionado no repositório.

6. Funcionalidades Principais

#	Funcionalidade	Destaque
RF-01	Landing Page	Countdown em tempo real, destaques do dia, atalhos de navegação.
RF-02	Tabela de Jogos	104 partidas, filtráveis por fase/grupo/data/seleção.
RF-03	Classificação e Grupos	Desempate FIFA completo, calculado automaticamente.
RF-04	Elencos das Seleções	48 seleções, busca e navegação entre perfis.
RF-05	Escalação do Brasil	Campo interativo, 7 formações, exportação de imagem.
RF-06	Bolão e Chaveamento	Simulação completa até o campeão, múltiplos bolões.
RF-07	Painel Administrativo	Atualização de placares, status e elencos.

Todas as sete funcionalidades definidas no PRD original foram entregues integralmente, com critérios de aceite conferidos manualmente em cada sprint correspondente.

7. Qualidade e Testes

Não há uma suíte de testes automatizados (Jest/Pytest) configurada — o plano de projeto pedia verificação manual dos fluxos críticos, feita via navegador real (automação Playwright) e chamadas diretas à API. O que foi validado:

- **Classificação:** cenário completo de grupo e um desempate por confronto direto construído propositalmente (dois times empatados em pontos/saldo/gols, decidido pelo resultado entre eles) — bateu exatamente com o esperado.
- **Bolão:** simulação dos 72 jogos de grupo + 32 do mata-mata até o campeão, testada via script de ponta a ponta na API e via interface real.
- **Escalação:** as 7 formações táticas confirmadas com 11 posições corretamente distribuídas em cada uma.
- **Countdown:** contagem testada até zero segundos, com a transição para "A Copa começou!" confirmada no momento exato.
- **Responsividade:** todas as páginas principais testadas em viewport mobile (375×812).
- **Backup do banco:** gerado, restaurado e os dados conferidos íntegros.

Limitação honesta: o teste "mobile" foi feito com o motor Chromium em viewport reduzido, não em dispositivos iOS Safari ou Chrome Android reais — este ambiente de desenvolvimento não tem acesso a esses dispositivos.

8. Deploy e Infraestrutura

Fiel ao princípio de "nenhum Docker", o projeto documenta duas rotas de deploy: um guia self-hosted para servidor Linux convencional (local ou VPS) e, mais recentemente, a publicação experimental do backend na Vercel.

Backend (self-hosted)

Serviço systemd rodando um único worker Uvicorn — SQLite não paraleliza escritas entre processos, então múltiplos workers só aumentariam o risco de "database is locked" sem ganho real de performance. O modo WAL do SQLite (ativado em produção) permite leituras concorrentes durante uma escrita, que é o ganho de concorrência efetivamente disponível aqui.

Frontend

Build estático gerado pelo Vite, servido via nginx com fallback de SPA e proxy reverso de `/api/` para o backend, com compressão gzip para os assets.

Backup e variáveis de ambiente

Script de backup usa a API nativa de cópia segura do SQLite (funciona com o banco em uso), compacta em `.gz` e mantém uma rotação dos backups mais recentes — pensado para ser agendado via cron. Um arquivo de exemplo documenta as variáveis de ambiente esperadas em produção (chave de administrador forte, origem CORS do domínio real).

Backend na Vercel: build e runtime

O backend também foi publicado na Vercel — frontend e backend como dois projetos separados —, o que revelou dois problemas reais em sequência. Primeiro, o build falhava: a Vercel resolve as dependências Python do backend com `uv` a partir do `pyproject.toml`, que não tinha uma tabela `[project]` válida ("No project table found"; correção detalhada na seção 5). Depois do build passar, o runtime falhava: funções serverless do Vercel rodam num sistema de arquivos somente leitura (fora de `/tmp`, que é efêmero), e qualquer rota do backend que toca o banco retorna 500 com "unable to open database file" — confirmado testando o backend já deployado. O SQLite do projeto precisa de um arquivo local gravável, incompatível com esse modelo de hospedagem.

Duas correções deixaram o que é possível funcionando, sem resolver a causa raiz: um `.env.production` no frontend define a URL real do backend (builds de produção do Vite carregam esse arquivo automaticamente), e o CORS do backend passou a aceitar o domínio do frontend por padrão. O backend em si segue precisando de uma hospedagem com disco persistente — VPS, Render, Railway ou Fly.io — para funcionar de fato; essa migração é uma decisão em aberto do responsável pelo projeto.

9. Estrutura do Repositório

```
backend/  
  app/  
    routes/      endpoints da API (jogos, grupos, selecoes, escalacoes, boloes, admin)  
    services/    regras de negócio (classificacao.py, chaveamento.py)  
  db/  
    migrations/  schema versionado  
    seeds/       dados iniciais (selecoes, jogos, jogadores)  
  scripts/       backup_db.py  
  main.py        entry point FastAPI  
  
frontend/  
  src/  
    components/  organizados por domínio (jogos, grupos, escalacao, bolao, admin...)  
    pages/       uma página por rota  
    hooks/       acesso a dados (useJogos, useBoloes, usePalpites...)  
    store/       estado global (Zustand)  
    utils/       formatação de datas, chamadas de API, formações táticas  
  
deploy/         templates de systemd e nginx para produção
```

10. Status Final

As nove sprints planejadas — da fundação ao deploy — foram concluídas integralmente. Cada uma foi implementada, testada em navegador real e documentada antes de seguir para a próxima.

Sprint	Entrega	Status
Sprint 0	Fundação e Setup	Concluída
Sprint 1	Landing Page	Concluída
Sprint 2	Tabela de Jogos	Concluída
Sprint 3	Classificação e Grupos	Concluída
Sprint 4	Elencos das Seleções	Concluída
Sprint 5	Escalação da Seleção Brasileira	Concluída
Sprint 6	Bolão e Simulador de Chaveamento	Concluída
Sprint 7	Painel Admin e Polimento	Concluída
Sprint 8	Testes e Deploy	Concluída

O código-fonte completo, com o histórico de commits sprint a sprint, está disponível publicamente em github.com/fernandojsmelo/copa2026.

Documento gerado como parte da entrega do projeto — Copa do Mundo 2026 App.